

AFC Ticketing

Volledige technische documentatie, architectuur en reconstructiegids

Gebaseerd op de codebase `ticketing` (Next.js 16 / React 19 / SQLite)

13 september 2026

Samenvatting

Dit document beschrijft de AFC Gent ticketing-applicatie tot op reconstructieniveau. Het legt uit *waarom* elke architectuurkeuze gemaakt is, *wat* ieder bestand doet, *hoe* de data laag, authenticatie, tickets, API's en gebruikersflows werken, en *hoe* je de tool vanaf nul opnieuw zou bouwen. De applicatie is een monolithische Next.js 16 App Router-app met ingebouwde API-routes, een lokale SQLite-database via **better-sqlite3**, optionele SMTP-mail, en HMAC-ondertekende QR-tickets die ook op ephemeral serverless-schijven blijven werken.

Inhoudsopgave

1	Doel, scope en ontwerpprincipes	4
1.1	Wat de tool doet	4
1.2	Niet-functionele eisen die de architectuur sturen	4
1.3	Waarom TypeScript	4
1.4	Waarom Next.js 16 (App Router) + React 19	5
1.5	Waarom SQLite + better-sqlite3	5
1.6	Kernstack	5
2	Repositorystructuur — elk bestand uitgelegd	6
2.1	Overzicht boom	6
2.2	Rootconfiguratie	6
2.2.1	package.json	6
2.2.2	tsconfig.json	6
2.2.3	next.config.ts	6
2.2.4	postcss.config.mjs + Tailwind 4	6
2.2.5	eslint.config.mjs	7
2.2.6	proxy.ts	7
2.2.7	AGENTS.md / README.md	7
2.3	Map lib/ — domeinlaag	7
2.3.1	lib/config.ts — omgeving en seeddefinities	7
2.3.2	lib/session.ts — sessietoken	7
2.3.3	lib/auth.ts	7
2.3.4	lib/ticket-token.ts — ondertekende tickets	7
2.3.5	lib/ticket.ts	8
2.3.6	lib/qr.ts	8
2.3.7	lib/validation.ts	8
2.3.8	lib/food.ts	8
2.3.9	lib/types.ts	8
2.3.10	lib/datetime.ts	8
2.3.11	lib/csv.ts	8
2.3.12	lib/email.ts	8
2.3.13	lib/db.ts — hart van de data laag	9
2.4	Map app/ — routes en UI	9
2.4.1	Layout & globale stijl	9
2.4.2	Publieke pages	9
2.4.3	Admin pages	9
2.5	Map components/	9
2.6	Map public/	10
2.7	Map data/ (runtime)	10
3	Datamodel en persistentie	11
3.1	Initialisatievolgorde bij cold start	11

3.2	Tabel events	11
3.2.1	Acceptatieregel inschrijving	11
3.3	Tabel registrations	12
3.4	Migratie single-event → multi-event	12
3.5	Bestanden op schijf	12
3.6	Domain mapping	12
4	Authenticatie en autorisatie	13
4.1	Admin login flow	13
4.2	Beschermingslagen (defense in depth)	13
4.3	Beperkingen van dit authmodel	13
5	API-referentie	14
5.1	Publiek	14
5.1.1	POST /api/register	14
5.1.2	GET /api/event-image/[id]	14
5.2	Auth	14
5.2.1	POST /api/auth	14
5.3	Admin — check-in & export	14
5.3.1	POST /api/checkin	14
5.3.2	GET /api/export?event=slug?	15
5.3.3	GET /api/cv/[id]	15
5.4	Admin — events /api/admin/events	15
5.5	Admin — attendees /api/admin/attendees	15
6	End-to-end flows	16
6.1	Publieke inschrijving	16
6.2	Deurcheck-in	16
6.3	Admin event met foto + lange beschrijving	16
6.4	Export	17
7	UI-architectuur en designkeuzes	18
7.1	Server vs client	18
7.2	Visuele taal	18
7.3	Waarom geen aparte componentlibrary	18
8	Omgeving, deploy en operationele concerns	19
8.1	Environment variables	19
8.2	Scripts	19
8.3	VPS (aanbevolen productie)	19
8.4	Vercel-beperkingen	19
8.5	Git-hygiëne	19
9	Reconstructiegids: bouw de tool opnieuw	20
9.1	Fase 0 — scaffold	20
9.2	Fase 1 — data laag	20
9.3	Fase 2 — tokens & validatie	20
9.4	Fase 3 — publieke API + UI	20
9.5	Fase 4 — admin	20
9.6	Fase 5 — harding	20
9.7	Acceptatiechecklist	21
10	Beslissingen, trade-offs en uitbreidingspaden	22

10.1	Belangrijkste trade-offs	22
10.2	Mogelijke evoluties (niet geïmplementeerd)	22
11	Appendix	23
11.1	Snelle bestand-naar-verantwoordelijkheid mapping	23
11.2	HTTP-statuscodes die de API consequent gebruikt	23
11.3	Slot	24

Hoofdstuk 1

Doel, scope en ontwerpprincipes

1.1 Wat de tool doet

De applicatie ondersteunt meerdere events binnen één installatie:

- Publieke homepage met openstaande events (optioneel met foto).
- Inschrijfformulier per event (naam, e-mail, telefoon, voedselvoorkeur, optionele extra info, verplicht CV als PDF).
- Ticketpagina met QR-code na inschrijving; optionele bevestigingsmail.
- Adminportaal: login, dashboard, eventbeheer (CRUD/archief/foto/uitgebreide beschrijving), deelnemersbeheer (toevoegen/verwijderen/check-in), QR-scanner, CSV-export.
- Archiefpagina voor afgelopen events.

1.2 Niet-functionele eisen die de architectuur sturen

Eén process / één service

Geen aparte backend-API-server. Alles leeft in Next.js (`app/` voor UI, `app/api/` voor HTTP). Dit vermindert deployment-complexiteit voor een clubtool.

Locale persistentie

SQLite in `data/tickets.db` + uploads in `data/uploads/`. Geen Postgres-vereiste. Op Vercel is de schijf read-only buiten `/tmp`; daarom schrijft de runtime daar naartoe.

Serverless-robust tickets

Op Vercel heeft elke serverless-instance een eigen lege `/tmp`-SQLite. Daarom zijn tickets HMAC-ondertekend: de payload zelf bewijst geldigheid, zelfs als de lokale DB de rij mist.

Admin is “good enough”, niet enterprise-IAM

Eén gedeeld wachtwoord + cookie-sessie. Geschikt voor een organisatiesleutel, niet voor multi-user RBAC.

Nederlandstalige UX

Copy, foutmeldingen, CSV-headers en datums (nl-BE, tijdzone Europe/Brussels).

1.3 Waarom TypeScript

- **Contracten over grenzen heen:** `FormData` → `Zod` → `DB-row` → `camelCase` domain model. TypeScript vangt mismatch tussen `RegistrationRow` (snake_case SQL) en `Registration` (app-model).

- **Next.js App Router** is TypeScript-first (`params: Promise<...>`, route handlers, server/client boundary).
- **Native modules** zoals `better-sqlite3` hebben `@types`; compile-time checks vermijden runtime “undefined column”.
- **Refactorveiligheid** bij multi-event migratie: slug, `event_id`, signed claims, en admin FormData-velden blijven synchroon.

JavaScript zou werken, maar de kost van types is laag t.o.v. de kost van stille datafouten bij check-in en tickets.

1.4 Waarom Next.js 16 (App Router) + React 19

- Server Components voor data-reads dicht bij SQLite (homepage, adminlijsten) zonder aparte JSON-BFF.
- Client Components alleen waar nodig (formulieren, scanner, interactieve tabellen).
- Route Handlers als API zonder Express.
- `proxy.ts` (Next 16-conventie) beschermt `/admin/*` vóór page-render.

Belangrijk: deze Next-versie wijkt af van oudere docs. Lees altijd `node_modules/next/dist/docs/` in de repo vóór framework-wijzigingen (`AGENTS.md`).

1.5 Waarom SQLite + better-sqlite3

- Zero-ops voor een club: geen managed DB, backups = map `data/` kopiëren.
- `better-sqlite3` is synchroon en snel in Node — ideaal in route handlers zonder connection pool.
- Nadelen bewust aanvaard: slechte fit voor multi-region serverless writes; Vercel-demo’s zijn ephemeral. Productie op een VPS met persistente schijf is het bedoelde pad.

1.6 Kernstack

Laag	Keuze
Runtime	Node 22, Next.js 16.3.1 (Turbopack in dev)
UI	React 19.2, Tailwind CSS 4, Google font Josefin Sans
Validatie	Zod 4
DB	better-sqlite3 13, bestand <code>data/tickets.db</code>
QR	<code>qrcode</code> (server), <code>@yudiel/react-qr-scanner</code> (admin camera)
Mail	nodemailer (optioneel SMTP)
Auth	SHA-256 sessiecookie <code>gate_session</code>
Tickets	HMAC-SHA256 tokenformaat <code>t1.<body>.<sig></code>

Hoofdstuk 2

Repositorystructuur — elk bestand uitgelegd

2.1 Overzicht boom

```
ticketing/  
  app/           # App Router: pages + API  
  components/    # React UI (server/client)  
  lib/           # Domeinlogica, DB, auth, mail, tokens  
  public/        # Statische assets (logo)  
  data/          # Runtime: tickets.db + uploads/ (niet 'broncode')  
  proxy.ts       # Admin route guard  
  next.config.ts  
  package.json  
  tsconfig.json  
  AGENTS.md / README.md
```

2.2 Rootconfiguratie

2.2.1 package.json

Definieert scripts `dev`, `build`, `start`, `lint` en alle dependencies. Er is **geen** testscript: kwaliteit wordt manueel/via `lint+build` afgedwongen.

2.2.2 tsconfig.json

Strict TypeScript, path alias `@/*` → projectroot. Daardoor imports als `@/lib/db` in plaats van relatieve `../../..`.

2.2.3 next.config.ts

Markeert native/CJS-modules (`better-sqlite3`, evt. `nodemailer`) correct voor bundling, zodat server routes in de Node-runtime blijven.

2.2.4 postcss.config.mjs + Tailwind 4

PostCSS laadt `@tailwindcss/postcss`. Styling zit grotendeels in utility-classes plus CSS-variabelen in `app/globals.css` (`-paper`, `-ink/forest`, `-accent`, `-muted`, `-card`, `-line`).

2.2.5 eslint.config.mjs

eslint-config-next: App Router-regels + TypeScript.

2.2.6 proxy.ts

Next 16 proxy/middleware-equivalent. Matcher: `/admin` en `/admin/:path*`.

- Niet ingelogd op adminpagina → redirect `/admin/login?next=...`
- Wel ingelogd op loginpagina → redirect `/admin`
- **API-routes** (`/api/*`) vallen *niet* onder deze matcher; elke admin-API herhaalt cookiecheck zelf.

2.2.7 AGENTS.md / README.md

Operationele notities voor agents/developers: defaults, geen verplichte `.env`, waarschuwing om `node_modules/`, `data/`, `.next/` niet per ongeluk te committen (er is geen `.gitignore` in deze repo).

2.3 Map lib/ — domeinlaag

2.3.1 lib/config.ts — omgeving en seeddefinities

- `getEvents()` leest seed-events uit (prioriteit): `EVENTS_JSON` → legacy `EVENT_*` vars → hard-coded fallback (`afc-avond`, `afc-workshop`).
- `getAppUrl()` uit `APP_URL` (default `http://localhost:3000`).
- `getAdminPassword()` uit `ADMIN_PASSWORD` (default `admin123`).
- **Belangrijk:** dit is *seed/config*, niet de live eventbron. Runtime-UI leest events uit SQLite via `lib/db.ts`.

2.3.2 lib/session.ts — sessietoken

```
SESSION_COOKIE = "gate_session"
token = hex(SHA-256(UTF8("gate:" + adminPassword)))
```

Vergelijking gebeurt length-check + XOR-loop (timing-vriendelijker dan `===`). Er zit **geen** expiry in het token zelf; cookie `maxAge` (typisch 12u) bepaalt levensduur. **Keuze:** simpel, geen JWT-library, werkt in Edge/proxy via Web Crypto.

2.3.3 lib/auth.ts

Re-export van session helpers + `requireAdmin()` voor Server Components: ongeldige cookie → `redirect(/admin/login)`.

2.3.4 lib/ticket-token.ts — ondertekende tickets

Formaat: `t1.<base64url(JSON claims)>.<base64url(HMAC-SHA256(body))>`.

Claims (v: 1): `id`, `eventId`, `eventSlug`, `eventName`, `name`, `email`, `phone`, `extraInfo`, `foodPreference`, `cvOriginalName`, `createdAt`.

Secret: `TICKET_SECRET` → anders `ADMIN_PASSWORD` → anders `demo-ticket-secret`.

`verifyTicketToken` gebruikt `timingSafeEqual`. Bij succes bouwt het een `Registration`-achtig object met `cvStoredName = id + ".pdf"` en `checkedInAt = null` (check-in status komt altijd uit DB als die er is).

Waarom signed tokens? Op Vercel is DB-state per instance niet gedeeld. De QR moet toch verifieerbaar blijven; daarna kan `getRegistrationByToken` de claims in de lokale DB upserten.

2.3.5 lib/ticket.ts

- `ticketUrl(slug, token) → {APP_URL}/ticket/{slug}/{encodeURIComponent(token)}`.
- `parseTicketPayload(raw)` accepteert: prefixes `TICKET:/AFC:`, volledige ticket-URL's (`/ticket/` of `/t/`), raw signed `t1...`, of legacy tokens `[A-Za-z0-9_-]{16,}`.

Dit maakt de scanner tolerant voor “camera leest URL” versus “camera leest kale token”.

2.3.6 lib/qr.ts

Genereert QR die de *ticket-URL* encodeert (niet enkel de token), 360px, ECL M, kleuren navy/wit. `qrDataURL` voor UI; `qrPngBuffer` voor e-mail-CID.

2.3.7 lib/validation.ts

Zod-schema `registrationFields`: slug, naam (2–120), e-mail (lower+regex), telefoon (8–40), `extraInfo` (max 2000, default leeg), `foodPreference` ∈ `FOOD_OPTIONS`.

2.3.8 lib/food.ts

Enum/lijst voedselvoorkeuren + `foodLabel()`. Één bron voor formulier, validatie en CSV.

2.3.9 lib/types.ts

`Registration` (camelCase) ↔ `RegistrationRow` (SQL join-kolommen) via `mapRegistration`. Voorkomt dat UI per ongeluk `snake_case` verwacht.

2.3.10 lib/datetime.ts

- Weergave in Europe/Brussels met nl-BE.
- `brusselsLocalToIso`: zet `datetime-local` (Brussel-wandtijd) om naar UTC ISO via iteratieve offsetcorrectie (zonder extra TZ-library).
- `isoToBrusselsInput`: omgekeerd voor adminformulieren.

2.3.11 lib/csv.ts

Excel-vriendelijke CSV: UTF-8 BOM, delimiter `;`, alle cellen gequoted. Kolommen: Event, Naam, E-mail, Telefoon, Voedselvoorkeur, Extra info, CV, Ingeschreven op, Aanwezig, Check-in tijd, Ticketcode.

2.3.12 lib/email.ts

Als `SMTP_HOST` én `SMTP_FROM` ontbreken: mail wordt overgeslagen (`{sent:false, reason:"not-configured"}`). Anders nodemailer + inline QR (CID). Inschrijving faalt **niet** als mail faalt (best-effort).

2.3.13 lib/db.ts — hart van de data laag

Zie hoofdstuk 3. Bevat schema, migratie single→multi-event, seed, CRUD voor events/registrations, check-in, file cleanup, Vercel padlogica, en signed-token upsert bij tokenlookup.

2.4 Map app/ — routes en UI

2.4.1 Layout & globale stijl

- app/layout.tsx: lang="nl", Josefin Sans CSS-variable, metadata “AFC Ticketing”.
- app/globals.css: design tokens + utility helpers + printregels.
- app/not-found.tsx: generieke 404 (o.a. onbekend ticket/event).

2.4.2 Publieke pages

Pad	Bestand / rol
/	app/page.tsx — open events (<code>getOpenEvents</code>), optionele foto
/events/[slug]	inschrijfpagina: intro + optionele description + RegisterForm
/ticket/[eventSlug]/[token]	ticket + QR; query nieuw, mail
/afgelopen	gearchiveerde events (read-only)

Alle list/detail pages gebruiken `export const dynamic = "force-dynamic"` zodat SQLite-wijzigingen meteen zichtbaar zijn (geen statische cache van eventlijsten).

2.4.3 Admin pages

Pad	Rol
/admin/login	wachtwoordformulier (<code>LoginForm</code>)
/admin	dashboard stats + snelle links
/admin/events	<code>EventManager</code> CRUD
/admin/deelnemers	<code>AttendeeTable</code> (+ <code>?event=</code>)
/admin/scanner	<code>CheckinScanner</code>

Elke admin page (behalve login) start met `await requireAdmin()`.

2.5 Map components/

SiteHeader.tsx

Publieke header met AFC-logo + link naar organisatie/login.

AdminNav.tsx

Clientnav + logout (`DELETE /api/auth`).

LoginForm.tsx

POST `/api/auth` met password; volgt `?next=`.

RegisterForm.tsx

Client form → POST `/api/register` (multipart) → redirect ticket.

TicketCard.tsx

Toont gegevens + QR + print; waarschuwt als mail faalde.

EventManager.tsx

Admin eventform: intro, uitgebreide beschrijving, optionele foto, open/close venster, archiveren, verwijderen.

AttendeeTable.tsx

Zoeken, filteren, manueel toevoegen/verwijderen, check-in toggle, CV-link, export.

CheckinScanner.tsx

Camera + manuele token/e-mail check-in via `/api/checkin`.

2.6 Map public/

Relevant: `afc-gent-logo.svg`. Overige SVG's zijn Next-starterresten en niet functioneel nodig.

2.7 Map data/ (runtime)

```
data/  
  tickets.db          # SQLite hoofdbestand  
  tickets.db-wal/shm  # WAL sidecars (lokaal)  
  uploads/  
    {registrationId}.pdf  
    event-{eventId}.jpg|png|webp|gif
```

Op Vercel: zelfde structuur onder `/tmp/ticketing-data/`.

Hoofdstuk 3

Datamodel en persistentie

3.1 Initialisatievolgorde bij cold start

1. Bepaal data-dir (cwd/data of /tmp/ticketing-data).
2. Maak uploadmap aan.
3. Open SQLite; journal mode WAL (lokaal) of DELETE (Vercel).
4. `foreign_keys = ON`.
5. `ensureEventSchema (CREATE TABLE IF NOT EXISTS + ensureColumn)`.
6. Seed events als tabel leeg is (`getEvents()` uit config).
7. Migreer legacy single-event `registrations` indien nodig.
8. Zorg dat registrations-tabel + indexes bestaan.
9. Cache connectie op `globalThis` voor hot reload / warm serverless.

3.2 Tabel events

```
id TEXT PRIMARY KEY
slug TEXT NOT NULL UNIQUE
name TEXT NOT NULL
date TEXT NOT NULL          -- ISO timestamp
location TEXT NOT NULL
intro TEXT NOT NULL         -- korte tekst homepage
description TEXT NOT NULL DEFAULT '' -- enkel eventpagina
image_stored_name TEXT      -- optioneel bestand in uploads/
is_open INTEGER NOT NULL DEFAULT 1 -- handmatige schakelaar
archived INTEGER NOT NULL DEFAULT 0
registration_opens_at TEXT   -- optioneel ISO
registration_closes_at TEXT -- optioneel ISO
created_at TEXT NOT NULL
```

3.2.1 Acceptatieregel inschrijving

Een event accepteert inschrijvingen iff:

$$\neg \text{archived} \wedge \text{is_open} = 1 \wedge (\text{now} \geq \text{opens_at} \vee \text{opens_at} = \emptyset) \wedge (\text{now} \leq \text{closes_at} \vee \text{closes_at} = \emptyset).$$

Homepage toont enkel events die deze regel passeren (`getOpenEvents`).

3.3 Tabel registrations

```
id TEXT PRIMARY KEY
event_id TEXT NOT NULL REFERENCES events(id) ON DELETE CASCADE
name TEXT NOT NULL
email TEXT NOT NULL COLLATE NOCASE
phone TEXT NOT NULL
extra_info TEXT NOT NULL DEFAULT ''
food_preference TEXT NOT NULL
cv_original_name TEXT NOT NULL
cv_stored_name TEXT NOT NULL
ticket_token TEXT NOT NULL UNIQUE
checked_in_at TEXT          -- NULL = nog niet binnen
created_at TEXT NOT NULL
UNIQUE(event_id, email)
```

Indexes op `event_id`, `checked_in_at`, `created_at`.

3.4 Migratie single-event → multi-event

Oude DB's hadden `registrations` zonder `event_id`. Procedure:

1. Detecteer via `PRAGMA table_info`.
2. Rename naar `registrations_legacy` (met herstelpad als een vorige run crashte en de legacy-tabel bleef bestaan).
3. Maak nieuwe tabel; `INSERT OR IGNORE ... SELECT` met fallback `event_id` = oudste event.
4. Drop legacy.

Deze migratie is bewust idempotent/resilient: een achtergebleven `registrations_legacy` mag `npm run build/start` niet breken.

3.5 Bestanden op schijf

- CV: `{registrationId}.pdf`
- Eventfoto: `event-{eventId}.{ext}`
- Bij image replace/delete of event/registration delete: best-effort `unlink`.

3.6 Domain mapping

SQL join levert `event_slug/event_name`; `mapRegistration` zet om naar camelCase. UI en API werken bij voorkeur met het domain model; DB-helpers accepteren vaak beide lagen intern.

Hoofdstuk 4

Authenticatie en autorisatie

4.1 Admin login flow

1. UI: POST `/api/auth` JSON `{password}`.
2. Server vergelijkt met `getAdminPassword()`.
3. Bij succes: cookie `gate_session=SHA256("gate:"+password)`, `httpOnly`, `sameSite=lax`, `path=/`, `secure` als `APP_URL` met `https` begint, `maxAge` $\approx 12u$.
4. Logout: DELETE `/api/auth` verwijdert cookie.

4.2 Beschermingslagen (defense in depth)

1. `proxy.ts` blokkeert admin *pages*.
2. `requireAdmin()` in server pages (extra redirect).
3. Elke gevoelige API checkt opnieuw de cookie (want `/api` zit niet in proxy matcher).

Publiek blijven o.a. `/api/register` en `/api/event-image/[id]`. CV-download is admin-only.

4.3 Beperkingen van dit authmodel

- Geen per-user accounts, geen audit log van wie inlogde.
- Token verandert automatisch als adminwachtwoord verandert (alle sessies invalid).
- Geen CSRF-token; mitigatie via `SameSite=Lax` + same-origin form posts.
- Voor internet-facing productie: sterk wachtwoord via env, HTTPS, reverse proxy.

Hoofdstuk 5

API-referentie

Alle hieronder genoemde routes draaien in `runtime = "nodejs"` (nodig voor `better-sqlite3` en `filesystem`).

5.1 Publiek

5.1.1 POST /api/register

Multipart form: `eventSlug`, `name`, `email`, `phone`, `foodPreference`, `extraInfo?`, `cv` (PDF, verplicht, ≤ 5 MB).

Stappen:

1. Zod-parse $\rightarrow$ 400 bij fout.
2. Event bestaat + accepteert inschrijvingen $\rightarrow$ anders 404.
3. PDF-type/grootte $\rightarrow$ 400/413.
4. Unieke e-mail per event $\rightarrow$ anders 409.
5. `randomUUID()` als id; schrijf CV; mint HMAC-token; insert DB.
6. Best-effort mail.
7. JSON `{ok, eventSlug, token, emailSent}` (UI redirect naar ticket).

5.1.2 GET /api/event-image/[id]

Publiek. 404 als geen image. Stream bestand met content-type op extensie; cachebaar.

5.2 Auth

5.2.1 POST /api/auth

/ DELETE /api/auth Zie vorig hoofdstuk.

5.3 Admin — check-in & export

5.3.1 POST /api/checkin

Cookie verplicht. Body JSON kan bevatten: `payload` (scanruwe tekst), `token`, `id`, `email`, `eventSlug?`, `undo?`.

Gedrag:

- `undo:true` + `id`: clear `checked_in_at`.
- Anders resolve registratie; zet check-in timestamp als nog leeg.
- Ambiguous e-mail over events → 409 met matches.
- Onbekend → 404.
- Succes: naam, foodPreference, alreadyCheckedIn, eventName, ...

5.3.2 GET /api/export?event=slug?

CSV download; optioneel gefilterd op event.

5.3.3 GET /api/cv/[id]

Admin-only PDF stream met originele filename in `Content-Disposition`.

5.4 Admin — events /api/admin/events

Method	Body / effect
POST	multipart create (+ optionele image)
PUT	multipart update (id + velden + optionele image/clearImage)
PATCH	JSON {id, archived}
DELETE	JSON {id} cascade registrations + files

Velden o.a. name, slug?, date (datetime-local Brussel), location, intro, description, isOpen, archived, registrationOpensAt/ClosesAt, image, clearImage. Images: jpeg/png/webp/gif, max 5 MB.

5.5 Admin — attendees /api/admin/attendees

- **POST** multipart: zelfde velden als register; CV optioneel; mag ook op gesloten events; optioneel `sendEmail=1`.
- **DELETE** JSON {id} + CV cleanup.

Hoofdstuk 6

End-to-end flows

6.1 Publieke inschrijving

```
Browser GET /
  -> Server: getOpenEvents() from SQLite
  -> Cards (image? + intro)
User clicks event
  -> GET /events/{slug}
  -> intro + description? + RegisterForm
User submits multipart
  -> POST /api/register
  -> validate, store PDF, mint t1-token, insert row, maybe SMTP
  -> redirect /ticket/{slug}/{token}?nieuw=1&mail=0|1
Ticket page
  -> getRegistrationByToken (DB or verify+upsert)
  -> qrDataURL(ticketUrl)
  -> TicketCard
```

6.2 Deurcheck-in

```
Admin login -> /admin/scanner
Camera reads QR (URL or token)
  -> POST /api/checkin { payload }
  -> parseTicketPayload -> getRegistrationByToken -> set checked_in_at
UI shows name + food preference + alreadyCheckedIn?
```

6.3 Admin event met foto + lange beschrijving

```
/admin/events -> EventManager
Create/Edit form includes:
  intro (homepage), description (event page only),
  optional image upload / clearImage
POST/PUT /api/admin/events
  -> create/update row; save event-{id}.ext
Homepage shows image if image_stored_name set
Event page shows description if non-empty
```

6.4 Export

Admin klikt export → GET `/api/export` → CSV met BOM voor Excel.

Hoofdstuk 7

UI-architectuur en designkeuzes

7.1 Server vs client

Server Components lezen DB direct (geen waterfalls via client fetch voor initial paint). Client Components isoleren browser-API's (File input, camera, `useRouter.refresh()`).

7.2 Visuele taal

Navy #081C3C, accent oranje #EC6525, papierachtige achtergrond, grote serif/display headings via utility `.serif`, afgeronde kaarten ($\approx 28\text{px}$). Doel: herkenbaar AFC/organisatiegevoel zonder zwaar design system.

7.3 Waarom geen aparte componentlibrary

Voor deze scope (enkele tientallen schermen) zijn Tailwind + lokale componenten sneller dan shadow/MUI te onderhouden. Interactieve “cards” bestaan alleen als echte UI-containers (formulieren, eventkaarten), niet als decoratie.

Hoofdstuk 8

Omgeving, deploy en operationele concerns

8.1 Environment variables

Var	Rol / default
ADMIN_PASSWORD	admin login (default <code>admin123</code>)
APP_URL	absolute URL in QR/mail (default <code>localhost:3000</code>)
TICKET_SECRET	HMAC secret (valt terug op admin password)
EVENTS_JSON / EVENT_*	seed bij lege DB
SMTP_HOST, SMTP_FROM, ...	optionele mail
VERCEL/VERCEL_ENV	schakelt data-dir naar <code>/tmp</code>

Geen verplichte `.env` voor lokale demo.

8.2 Scripts

```
npm install
npm run dev      # http://localhost:3000
npm run lint
npm run build && npm start
```

8.3 VPS (aanbevolen productie)

Persistente schijf, Node 22, reverse proxy met HTTPS, env met sterk wachtwoord + `APP_URL` + SMTP. Backup: stop service, kopieer `data/`. Update: `git pull && npm ci && npm run build && systemctl restart ....`

8.4 Vercel-beperkingen

Geschikt als preview/demo. Schrijfacties in `/tmp` verdwijnen tussen cold starts. Signed tickets mitigeren ticketweergave; blijvende registraties vereisen persistente storage (dus VPS of externe object/DB store — niet in huidige code).

8.5 Git-hygiëne

Repo mist `.gitignore`. Commit nooit per ongeluk `node_modules/`, `.next/`, `data/tickets.db*`, of grote uploadbinaries.

Hoofdstuk 9

Reconstructiegids: bouw de tool opnieuw

9.1 Fase 0 — scaffold

1. Next.js App Router + TS + Tailwind 4 + ESLint.
2. Dependencies: better-sqlite3, zod, qrcode, nodemailer, react-qr-scanner + types.
3. Path alias @/*; proxy.ts voor /admin/*.

9.2 Fase 1 — datalaag

Implementeer lib/db.ts exact volgens schema hierboven, inclusief seed + migratie. Schrijf eerst helpers: createEvent, createRegistration, getRegistrationByToken (met verify+upsert), checkInRegistration.

9.3 Fase 2 — tokens & validatie

ticket-token.ts, ticket.ts, validation.ts, food.ts, datetime.ts.

9.4 Fase 3 — publieke API + UI

POST /api/register, homepage, eventpagina, ticketpagina, optionele mail/QR.

9.5 Fase 4 — admin

Auth cookie, dashboard, events CRUD (+ image/description), attendees, scanner, export, CV download.

9.6 Fase 5 — harding

force-dynamic op data pages; Vercel padlogica; image/CV size limits; unique email constraint; archiefvensters; resilient legacy migratie.

9.7 Acceptatiechecklist

- Inschrijven mét PDF werkt; duplicate e-mail → 409.
- Ticket QR opent dezelfde ticketpagina; scanner checkt in.
- Event zonder foto/beschrijving blijft netjes; mét foto/beschrijving toont correcte plaatsen.
- Zonder SMTP slaagt register toch.
- Admin delete event ruimt registrations + files op.
- `npm run lint` en `npm run build` groen.

Hoofdstuk 10

Beslissingen, trade-offs en uitbreidingspaden

10.1 Belangrijkste trade-offs

- **SQLite vs Postgres:** eenvoud vs horizontale schaal.
- **Signed tickets vs opaque DB tokens:** serverless-resilience vs revoke-from-DB-only. Huidige hybride: opaque unique kolom + HMAC claims.
- **Shared admin password vs echte users:** snelle ops vs accounting.
- **CV op lokale schijf vs S3:** eenvoud vs multi-instance deeling.
- **Geen testsuite:** snellere iteratie; regressierisico — overweeg later API integration tests.

10.2 Mogelijke evoluties (niet geïmplementeerd)

- Object storage voor CV's/images.
- Postgres + migrations framework.
- Rate limiting op `/api/register`.
- Per-scanner accounts / role-based access.
- Soft-delete en audit trail.
- Officiële `.gitignore` en `.env.example`.

Hoofdstuk 11

Appendix

11.1 Snelle bestand-naar-verantwoordelijkheid mapping

Bestand	Verantwoordelijkheid
lib/db.ts	Schema, CRUD, migratie, uploads pad
lib/config.ts	Env + seed events
lib/session.ts/auth.ts	Cookie auth
lib/ticket-token.ts	HMAC tickets
lib/ticket.ts/qr.ts	URL + QR
lib/validation.ts/food.ts	Inputcontracten
lib/email.ts/csv.ts/datetime.ts	Mail, export, tijd
proxy.ts	Admin page gate
app/api/register/route.ts	Publieke inschrijving
app/api/auth/route.ts	Login/logout
app/api/checkin/route.ts	Check-in/undo
app/api/export/route.ts	CSV
app/api/cv/[id]/route.ts	CV download
app/api/event-image/[id]/route.ts	Publieke eventfoto
app/api/admin/events/route.ts	Eventbeheer
app/api/admin/attendees/route.ts	Deelnemers add/delete
components/EventManager.tsx	Admin event UI
components/AttendeeTable.tsx	Admin deelnemers UI
components/CheckinScanner.tsx	Scanner UI
components/RegisterForm.tsx	Publiek formulier
components/TicketCard.tsx	Ticketweergave

11.2 HTTP-statuscodes die de API consequent gebruikt

- 200 — succes
- 400 — validatiefout / slechte input
- 401 — niet ingelogd
- 404 — onbekend event/ticket/deelnemer / gesloten inschrijving
- 409 — conflict (duplicate e-mail, ambiguous check-in)
- 413 — upload te groot

11.3 Slot

Wie dit document volgt, kan de AFC Ticketing-tool opnieuw opzetten met dezelfde architectuurkeuzes: monolithische Next.js-app, SQLite, HMAC-tickets, optionele SMTP, multi-event admin, en een bewuste scheiding tussen korte intro (homepage) en uitgebreide beschrijving + optionele foto (eventkaart/eventpagina).